

IX BOB. NUMPY-DA CHIZIQLI ALGEBRA (LINEAR ALGEBRA IN NUMPY)

Xos qiymatlar (**Eigenvalues**) va xos vektorlar (**Eigenvectors**) chiziqli algebraning fundamental tushunchalaridir. NumPy turli xil chiziqli algebra amallarini samarali bajarish uchun mustahkam `numpy.linalg` modulini taqdim etadi.

9.1. NumPy kutubxonasida chiziqli algebra (Linear Algebra in NumPy)

NumPy-ning Chiziqli algebra moduli har qanday numpy massivida chiziqli algebra amallarini qo'llash uchun turli metodlarni taklif etadi. Ushbu modul yordamida quyidagilarni topish mumkin:

- Massivning **rangi (rank)**, **determinanti**, **izi (trace)** va h.k.
- Matrisalarning **xos qiymatlari**.
- Matrisa va vektor **ko'paytmalari** (skalyar (dot), ichki (inner), tashqi (outer) ko'paytmalar va h.k.), matrisani **darajaga ko'tarish**.
- Chiziqli yoki tenzor **tenglamalarni yechish** va boshqalar!

Misol:

```
import numpy as np

A = np.array([[6, 1, 1],
              [4, -2, 5],
              [2, 8, 7]])

# Matrisaning rangi (Rank)
print("A ning rangi:", np.linalg.matrix_rank(A))

# Matrisa A ning izi (Trace - asosiy diagonal elementlari yig'indisi)
print("A ning izi:", np.trace(A))

# Matrisaning determinanti
print("A ning determinanti:", np.linalg.det(A))

# Matrisa A ning teskarisi (Inverse)
print("A ning teskarisi:\n", np.linalg.inv(A))
```

```
# Matrisa A ni 3-darajaga ko'tarish
print("A matrisasining 3-darajasi:\n", np.linalg.matrix_power(A, 3))

A ning rangi: 3
A ning izi: 11
A ning determinanti: -306.0
A ning teskarisi:
[[ 0.17647059 -0.00326797 -0.02287582]
 [ 0.05882353 -0.13071895  0.08496732]
 [-0.11764706  0.1503268   0.05228758]]
A matrisasining 3-darajasi:
[[336 162 228]
 [406 162 469]
 [698 702 905]]
```

Izoh:

- **Rank:** Chiziqli bog'liq bo'lmagan qatorlar yoki ustunlar soni.
- **Trace:** Matrisaning asosiy diagonali bo'ylab elementlar yig'indisi ($6 + (-2) + 7 = 11$).
- **Matrix Power:** Matrisani o'z-o'ziga berilgan marta ko'paytirish (bu elementma-element darajaga ko'tarishdan farq qiladi).

Matrisa va vektor ko'paytmalari

dot() funksiyasidan foydalanish: Bu funksiya ikki massivning skalyar ko'paytmasini qaytaradi. U 1D (vektorlar) va 2D (matrisalar) massivlar bilan birdek ishlaydi. 2D massivlarda foydalanilganda, u matrisalarni ko'paytirish amalini bajaradi.

N-o'lchamli massivlar uchun u a massivining oxirgi o'qi va b massivining oxiridan bitta oldingi o'qi bo'yicha ko'paytmalar yig'indisini hisoblaydi:

$$dot(a, b)[i, j, k, m] = \sum (a[i, j, :] \times b[k, :, m])$$

```
# Skalyar qiymatlar
prod = np.dot(5, 4)
print("Skalyar qiymatlar ko'paytmasi:", prod)

# 1D massiv (Kompleks sonlar)
a = 2 + 3j
b = 4 + 5j
```

```
res = np.dot(a, b)
print("Skalyar ko'paytma:", res)
```

Skalyar qiymatlar ko'paytmasi: 20

Skalyar ko'paytma: (-7+22j)

Izoh: $a = 2 + 3j$ $b = 4 + 5j$

Endi, skalyar ko'paytma: $= 2(4 + 5j) + 3j(4 + 5j)$
 $= 8 + 10j + 12j + 15j^2$ (bu yerda $j^2 = -1$) $= 8 + 10j + 12j - 15$
 $= -7 + 22j$

vdot() funksiyasidan foydalanish: Bu funksiya ham skalyar ko'paytmani qaytaradi, biroq bir muhim farqi bor: u ko'paytirishdan oldin birinchi argumentning **kompleks qo'shmasini** (complex conjugate) oladi. Bu xususiyat uni kompleks qiymatli massivlar bilan ishlashda juda foydali qiladi.

```
# 1D massiv
a = 2 + 3j
b = 4 + 5j
res = np.vdot(a, b)
print("vdot ko'paytma:", res)
```

vdot ko'paytma: (23-2j)

Izoh: $a = 2 + 3j$

$b = 4 + 5j$

Metodga ko'ra, a ning qo'shmasini olamiz: $2 - 3j$

Endi, skalyar ko'paytma:

$= (2 - 3j)(4 + 5j) = 2(4 + 5j) - 3j(4 + 5j) = 8 + 10j - 12j - 15j^2$
 $= 8 + 10j - 12j + 15 = 23 - 2j$

Matrisa va vektor ko'paytmalarining keng tarqalgan funksiyalari

Quyidagi jadvalda turli xil matrisa va vektor ko'paytmalarini bajarish uchun ishlatiladigan NumPy funksiyalari keltirilgan:

FUNKSIYA

TAVSIF

matmul()

Ikki massivning matrisa ko'paytmasi.

inner()

Ikki massivning ichki ko'paytmasi.

outer()

Ikki vektorning tashqi ko'paytmasini hisoblash.

FUNKSIYA

TAVSIF

`linalg.multi_dot()`

Eng tez hisoblash tartibini avtomatik tanlagan holda, bitta funksiya chaqiruvi bilan ikki yoki undan ortiq massivlarning skalyar ko'paytmasini hisoblash.

`tensordot()`

O'lchami ≥ 1 -D bo'lgan massivlar uchun belgilangan o'qlar bo'yicha tenzor skalyar ko'paytmasini hisoblash.

`einsum()`

Operandlarda Eynshteyn yig'indi konventsiyasini (Einstein summation convention) hisoblaydi.

`einsum_path()`

Oraliq massivlar yaratilishini hisobga olgan holda, `einsum` ifodasi uchun eng kam xarajatli qisqarish tartibini aniqlaydi.

`linalg.matrix_power()`

Kvadrat matrisani butun n darajaga ko'taradi.

`kron()`

Ikki massivning Kronecker ko'paytmasini hisoblaydi.

Asosiy farqlar haqida qisqacha izoh:

- `matmul()` vs `dot()`: `matmul()` funksiyasi skalyar ko'paytmani qo'llab-quvvatlamaydi va 3D/4D massivlar (batched matrices) bilan ishlashda `dot()` dan farqli (ko'proq standartlarga mos) ishlaydi.
- `outer()`: Agar sizda u va v vektorlari bo'lsa, tashqi ko'paytma natijasi $M_{i,j} = u_i \times v_j$ ko'rinishidagi matrisa bo'ladi.
- `kron()`: Bu amal blokli matrisalarni yaratishda ishlatiladi, bunda birinchi matrisaning har bir elementi ikkinchi matrisaning butun o'ziga ko'paytiriladi.

Tenglamalarni yechish va matrisalarni teskarilash

linalg.solve() funksiyasidan foydalanish: Bu funksiya $Ax = b$ ko'rinishidagi chiziqli tenglamalar tizimining aniq yechimini topish uchun ishlatiladi. Bu yerda:

- **A** — koeffitsientlar matrisasi.
- **b** — ozod hadlar (o'zgarmaslar) matrisasi.
- **x** — biz topmoqchi bo'lgan noma'lum o'zgaruvchi.

```
a = np.array([[1, 2], [3, 4]])
b = np.array([8, 18])
print("Chiziqli tenglamalar yechimi:", np.linalg.solve(a, b))
```

Chiziqli tenglamalar yechimi: [2. 3.]

linalg.lstsq() funksiyasidan foydalanish: Agar tizimning aniq yechimi bo'lmasa (masalan, u o'ta aniqlangan yoki ma'lumotlar shovqinli bo'lsa), ushbu funksiya bashorat qilingan va haqiqiy qiymatlar o'rtasidagi farqni minimallashtiradigan eng mos yechimni (**eng kichik kvadratlar usuli**) topadi.

```
import numpy as np
import matplotlib.pyplot as plt

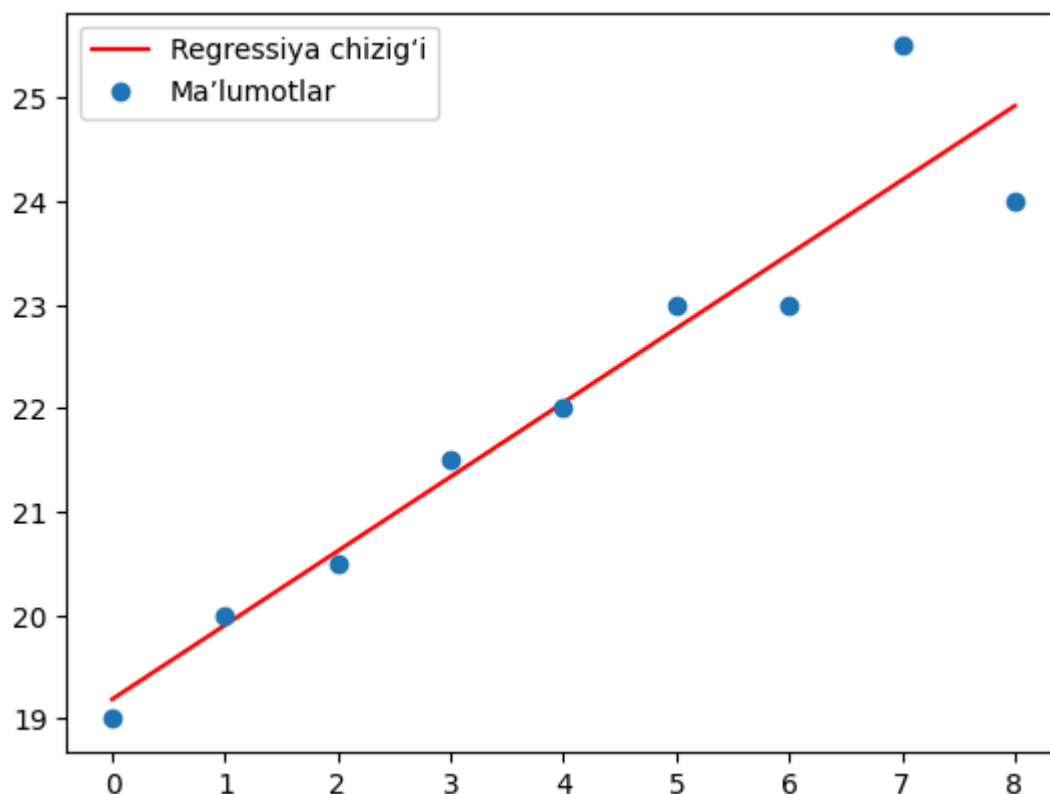
# x koordinatalari
x = np.arange(0, 9)
A = np.array([x, np.ones(9)])

# Chiziqli hosil qilingan ketma-ketlik (y koordinatalari)
y = [19, 20, 20.5, 21.5, 22, 23, 23, 25.5, 24]

# Regressiya chizig'i parametrlarini (w) olish
w = np.linalg.lstsq(A.T, y, rcond=None)[0]

# Regressiya chizig'ini qurish
line = w[0]*x + w[1]

plt.plot(x, line, 'r-', label='Regressiya chizig'i')
plt.plot(x, y, 'o', label='Ma'lumotlar')
plt.legend()
plt.show()
```



Izoh:

1. `np.linalg.solve()` : Faqat kvadrat matrisalar ($n \times n$) va yagona aniq yechim mavjud bo'lgan holatlar uchun mo'ljallangan.
2. `np.linalg.lstsq()` : Ko'pincha ma'lumotlarni tahlil qilishda va statistik modellashtirishda (chiziqli regressiya) eng mos chiziqni topish uchun ishlatiladi. U hatto yechim bo'lmagan hollarda ham "eng yaxshi yaqinlashish"ni qaytaradi.

Matrisalarni yechish va teskarilash uchun umumiy funksiyalar

Quyida matrisalarni yechish, ularning teskarisini yoki psevdoteskarisini hisoblash uchun eng foydali funksiyalar keltirilgan:

FUNKSIYA

TAVSIF

`numpy.linalg.tensorsolve()`

$a \times x = b$ tenzor tenglamasini x uchun yechadi.

`numpy.linalg.inv()`

Matrisaning (multiplikativ) teskarisini hisoblaydi.

FUNKSIYA

TAVSIF

`numpy.linalg.pinv()`

Matrisaning (Mur-Penrouz bo'yicha) psevdoteskari shaklini hisoblaydi.

`numpy.linalg.tensorinv()`

N-o'lchamli massivning "teskarisini" hisoblaydi.

NumPy Chiziqli Algebrasidagi maxsus funksiyalar

1. Determinantni topish — `numpy.linalg.det()`

Determinant — bu kvadrat matrisadan hisoblanadigan raqamdir. U matrisaning teskarisi mavjudligini aniqlashga yordam beradi va ko'pincha chiziqli tenglamalar tizimini yechishda ishlatiladi.

```
A = np.array([[6, 1, 1],
              [4, -2, 5],
              [2, 8, 7]])

print("A ning determinanti:", np.linalg.det(A))
```

A ning determinanti: -306.0

2. Izni topish — `numpy.trace()`

Matrisaning izi (trace) uning asosiy diagonalidagi elementlar yig'indisidir. U chiziqli algebra va statistikada tez-tez qo'llaniladi.

```
A = np.array([[6, 1, 1],
              [4, -2, 5],
              [2, 8, 7]])

print("A ning izi:", np.trace(A))
```

A ning izi: 11

Izoh: Hisoblash jarayoni: $6 + (-2) + 7 = 11$.

Umumiy maxsus funksiyalar

Quyida NumPy-da mavjud bo'lgan boshqa ixtisoslashgan chiziqli algebra funksiyalari ro'yxati keltirilgan:

FUNKSIYA

TAVSIF

`numpy.linalg.norm()`

Matrisa yoki vektor normasi.

`numpy.linalg.cond()`

Matrisaning shartlilik sonini (condition number) hisoblash.

`numpy.linalg.matrix_rank()`

SVD metodi yordamida massivning matrisa rangini qaytarish.

`numpy.linalg.cholesky()`

Xoleskiy (Cholesky) yoyilmasi.

`numpy.linalg.qr()`

Matrisaning QR faktorializatsiyasini hisoblash.

`numpy.linalg.svd()`

Singulyar qiymatlar bo'yicha yoyilma (SVD).

NumPy-da matrisaning xos qiymatlari funksiyalari

NumPy o'zining `linalg` (Chiziqli algebra) moduli tarkibida matrisalarning xos qiymatlari va xos vektorlarini hisoblash uchun maxsus funksiyalarni taqdim etadi.

`linalg.eigh()` funksiyasidan foydalanish: Bu funksiya **Ermit (Hermitian)** (kompleks simmetrik) yoki **haqiqiy simmetrik** matrisalar uchun ishlatiladi. U ikkita qiymat qaytaradi:

1. Xos qiymatlar massivi.
2. Xos vektorlar matrisasi (har bir ustun bitta xos qiymatga mos keladi).

```
import numpy as np
from numpy import linalg as geek

# array funksiyasi yordamida massiv yaratish
a = np.array([[1, -2j], [2j, 5]])
print("Massiv:\n", a)

# eigh() funksiyasi yordamida xos qiymat va xos vektorlarni hisoblash
c, d = geek.eigh(a)
print("Xos qiymatlar:", c)
print("Xos vektorlar:\n", d)
```


Massiv:

```
[[ 1.+0.j -0.-2.j]
 [ 0.+2.j  5.+0.j]]
```

Xos qiymatlar: [0.17157288 5.82842712]

Xos vektorlar:

```
[[ -0.92387953+0.j          -0.38268343+0.j          ]
 [  0.           +0.38268343j  0.           -0.92387953j]]
```

`linalg.eig()` funksiyasidan foydalanish: Bu funksiya **umumiy kvadrat matrisalar** (simmetrik bo'lishi shart emas) uchun ishlatiladi. U ham xos qiymatlarni, ham xos vektorlarni qaytaradi.

```
import numpy as np
from numpy import linalg as geek

# diag funksiyasi yordamida diagonal matrisa yaratish
a = np.diag((1, 2, 3))
print("Massiv:\n", a)

# eig() funksiyasi yordamida xos qiymat va xos vektorlarni hisoblash
c, d = geek.eig(a)
print("Xos qiymatlar:", c)
print("Xos vektorlar:\n", d)
```

Massiv:

```
[[1 0 0]
 [0 2 0]
 [0 0 3]]
```

Xos qiymatlar: [1. 2. 3.]

Xos vektorlar:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Xos qiymatlar bo'yicha umumiy funksiyalar

Ushbu jadvalda xos qiymat va xos vektorlarni hisoblash bilan bog'liq turli funksiyalar umumlashtirilgan:

FUNKSIYA

TAVSIF

`linalg.eigvals()`

Umumiy matrisaning faqat xos qiymatlarini hisoblaydi.

`linalg.eigvalsh(a[, UPL0])`

Kompleks Ermit yoki haqiqiy simmetrik matrisaning faqat xos qiymatlarini

hisoblaydi.

9.2. QR Dekompozitsiyasi (QR Factorization of the NumPy Array)

Berilgan NumPy massivining QR yoyilmasini (factorization) olish.

Ushbu maqolada biz matrisaning **QR dekompozitsiyasi** yoki **QR yoyilmasini** ko'rib chiqamiz. Matrisaning QR yoyilmasi — bu A matrisasini $A = QR$ ko'rinishiga keltirishdir. Bu yerda Q — ortogonal matrisa, R esa yuqori uchburchakli (upper-triangular) matrisadir. Biz matrisani `numpy.linalg.qr()` funksiyasi yordamida yoyilmalarga ajratamiz.

Sintaksis

```
numpy.linalg.qr(a, mode='reduced')
```

Parametrlar:

- **a**: Faktotsiyalanishi (yoyilishi) kerak bo'lgan (M, N) o'lchamli matrisa.
- **mode**: Ixtiyoriy parametr. U `reduced`, `complete`, `r` yoki `raw` bo'lishi mumkin.

Kod misollari

1-misol: 2×2 o'lchamli matrisaning QR yoyilmasi

```
# NumPy massivini yaratamiz
arr = np.array([[10, 22], [13, 6]])

# Massivning QR faktorini topamiz
q, r = np.linalg.qr(arr)

# Natijani chiqaramiz
print("Matrisa dekompozitsiyasi:")
print("q=\n", q)
print("r=\n", r)
```

Matrisa dekompozitsiyasi:

```
q=  
[[-0.60971076 -0.79262399]  
 [-0.79262399  0.60971076]]  
r=  
[[-16.40121947 -18.16938067]  
 [  0.          -13.7794632  ]]
```

2-misol: 4×2 o'lchamli matrisaning QR yoyilmasi

```
# NumPy massivini yaratamiz  
arr = np.array([[0, 1], [1, 0], [1, 1], [2, 2]])  
  
# Massivning QR faktorini topamiz  
q, r = np.linalg.qr(arr)  
  
# Natijani chiqaramiz  
print("Matrisa dekompozitsiyasi:")  
print("q=\n", q)  
print("r=\n", r)
```

Matrisa dekompozitsiyasi:

```
q=  
[[ 0.          0.73854895]  
 [-0.40824829 -0.61545745]  
 [-0.40824829  0.12309149]  
 [-0.81649658  0.24618298]]  
r=  
[[-2.44948974 -2.04124145]  
 [ 0.          1.3540064  ]]
```

Izoh: QR yoyilmasi chiziqli tenglamalar tizimini yechishda, eng kichik kvadratlar usulida va matrisaning xos qiymatlarini hisoblash algoritmlarida (masalan, QR algoritmi) keng qo'llaniladi.

Q matrisasining xususiyati shundaki, uning ustunlari o'zaro ortogonaldir (ya'ni $Q^T Q = I$), **R** matrisasining esa asosiy diagonal ostidagi barcha elementlari nolga teng bo'ladi.

3-misol: 3×3 o'lchamli matrisaning QR yoyilmasi

Ushbu misolda butun sonli elementlardan iborat 3×3 o'lchamli matrisa QR faktorizatsiya qilinadi.

```
# NumPy massivini (matrisa) yaratamiz  
arr = np.array([[5, 11, -15],  
                [12, 34, -51],  
                [-24, -43, 92]], dtype=np.int32)
```

```
# Massivning QR faktorlarini topamiz
q, r = np.linalg.qr(arr)

# Natijani chiqaramiz
print("Matrisa dekompozitsiyasi:")
print("q=\n", q)
print("r=\n", r)
```

Matrisa dekompozitsiyasi:

q=

```
[[-0.18318583 -0.08610905  0.97929984]
 [-0.43964598 -0.88381371 -0.15995231]
 [ 0.87929197 -0.45984624  0.12404465]]
```

r=

```
[[-27.29468813 -54.77256208 106.06459346]
 [  0.          -11.22347731  4.06028083]
 [  0.           0.          4.88017756]]
```

Izoh: Natijadan ko‘rinib turibdiki, **R** matrisasi yana yuqori uchburchakli shaklga ega (diagonal ostidagi barcha elementlar nolga yoki nolga juda yaqin songa teng). **Q** matrisasi esa ortogonaldir, ya’ni uning ustunlari birlik uzunlikka ega va o‘zaro perpendikulyar. Agar siz `np.dot(q, r)` amalini bajarsangiz, qaytadan asl `arr` matrisasini (yaxlitlash xatoliklarini hisobga olgan holda) hosil qilasiz.

9.3. Xos sonlarni hisoblash (Computing the Eigenvalues)

NumPy-dagi `np.linalg.eigvals()` metodi kvadrat matrisaning faqat xos qiymatlarini (eigenvalues) hisoblash uchun ishlatiladi. `eig()` metodidan farqli o‘laroq, u xos vektorlarni hisoblab o‘tirmaydi, bu esa resurslarni tejash imkonini beradi.

Sintaksis: `numpy.linalg.eigvals(a)`

Parametrlar:

- **a:** Xos qiymatlari hisoblanishi kerak bo‘lgan 2 o‘lchamli kvadrat matritsa.

Qaytaradi:

- Xos qiymatlardan iborat 1 o‘lchamli NumPy massivi (haqiqiy yoki kompleks sonlar).

Kod misollari

1-misol: Oddiy 2×2 o'lchamli matrisaning xos qiymatlarini hisoblash.

```
from numpy import linalg as LA  
  
val = LA.eigvals([[1, 2], [3, 4]])  
print(val)
```

```
[-0.37228132  5.37228132]
```

Izoh: `[[1, 2], [3, 4]]` matrisasi ikkita xos qiymatga ega: -0.37 va 5.37. Ushbu qiymatlar matrisa o'z xos vektorlariga ta'sir qilgandagi masshtab koeffitsientlarini ko'rsatadi.

2-misol: 3×3 o'lchamli matrisaning xos qiymatlarini hisoblash.

```
val = LA.eigvals([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
print(val)
```

```
[ 1.61168440e+01 -1.11684397e+00 -1.30367773e-15]
```

Izoh: Ushbu 3×3 matrisa uchta xos qiymatga ega: 16.11, -1.11 va 0 ga juda yaqin bo'lgan juda kichik qiymat (suzuvchi nuqtali yaxlitlash xatoligi tufayli).

3-misol: Simmetrik matrisaning xos qiymatlari. Simmetrik matrisalarning xos qiymatlari har doim haqiqiy sonlar bo'ladi.

```
val = LA.eigvals([[2, -1], [-1, 2]])  
print(val)
```

```
[3. 1.]
```

Izoh: `[[2, -1], [-1, 2]]` simmetrik matrisasi uchun xos qiymatlar 3 va 1 ga teng. Simmetrik matrisalar haqiqiy xos qiymatlarni kafolatlaydi, shuning uchun natijada kompleks qism mavjud emas.



Nazorat savollari

1-daraja: Asosiy tushunchalar

1. NumPy-da chiziqli algebra amallari uchun asosan qaysi moduldan foydalaniladi?
2. Ikki massivning skalyar ko'paytmasini hisoblashda `np.dot()` funksiyasi qanday vazifani bajaradi?
3. Matritsaning izi (`trace`) nima va u NumPy-da qanday hisoblanadi?
4. Vektor yoki matritsa normasini (`np.linalg.norm()`) hisoblash amaliyotda nima uchun kerak?

2-daraja: Funktsiyalarning ishlash mexanizmi

5. `np.inner()` va `np.outer()` funksiyalari o'rtasidagi asosiy farqni tushuntirib bering.
6. Matritsani darajaga ko'tarishda `np.linalg.matrix_power()` funksiyasi qanday ishlaydi?
7. Matritsaning rangi (`rank`) nima va uni aniqlash uchun NumPy-da qaysi funksiya ishlatiladi?
8. `np.linalg.eigvals()` funksiyasi `np.linalg.eig()` dan qanday jihati bilan farq qiladi?

3-daraja: Amaliy tahlil va murakkab algoritmlar

9. Chiziqli tenglamalar sistemasini yechishda `np.linalg.solve()` funksiyasidan foydalanish shartlari qanday?
10. Teskari matritsani (`np.linalg.inv()`) hisoblashda matritsaning determinanti nolga teng bo'lsa, nima sodir bo'ladi?
11. Simmetrik matritsalarining xos qiymatlari boshqa turdagi matritsalaridan qanday xususiyati bilan ajralib turadi?
12. Ma'lumotlar tahlilida xos qiymatlar (eigenvalues) va xos vektorlarning ahamiyatini izohlang.